

Chapter 7

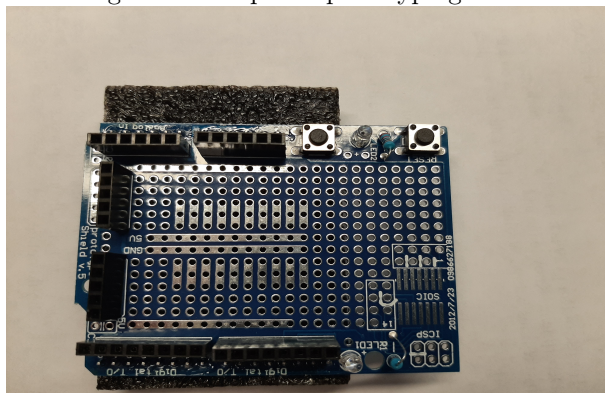
DataLogging

It today's lab, we are going to measure temperature. We will use a transducer that turns temperature (energy of the air molecules around us) into a voltage. "Transducer" is a general word for a device that turns some energy measurement into a voltage (really any form of energy to a different form of energy). But really our temperature measurement is just to have something useful to measure. We will take a break from testing physics models today. And we will learn about a second way to collect data so that we can analyze it on a computer.

Often we need to have a data collection device that can operate far from our computer, but still save data for later use. For example, if you were to launch your Arduino-instrument on a high altitude balloon.

Today we all need to be able to have our Arduino collect data and save it with no computer attached. We will do this using a shield. An Arduino shield fits over the top of your Arduino, connecting to the necessary pins to do its job. The other pins are still available to use for other measurements.

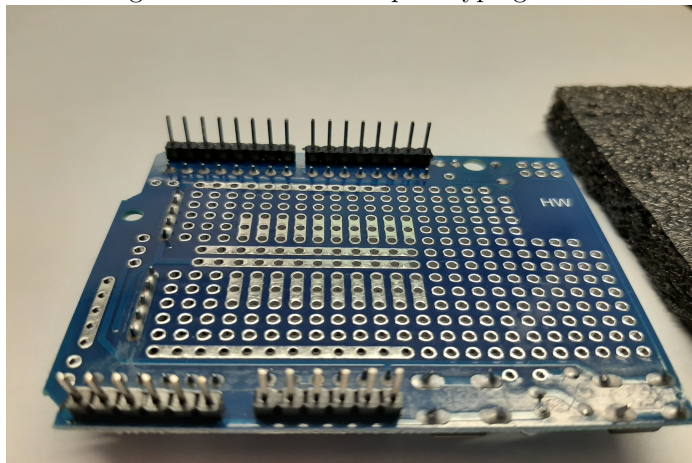
Figure 7.1: Top of a prototyping shield



7.1 Arduino Shields

Arduino shields are a simple way to add specific functionality to your Arduino. There are many different shields with many different features. Some shields have additional integrated circuit (IC) chips on them that can add additional computational power. Others are extremely simple and are designed so you can have a more solid electrical connections with your experiment. The previous figure and the next figure are the top and bottom of an example of a simple prototyping shield.

Figure 7.2: Bottom of a prototyping shield



Notice the long wire pins on the bottom. These are designed to be plugged in directly into the Arduino. When not in use these should be pushed into the conductive foam. This will not only protect them from getting bent, but will also protect the electrical circuits on the shield from static shocks.

Many shields are compatible with other shields such that they can be stacked onto each other. This will depend on what pins each shield is using and on the physical placement of both the bottom and top pins.

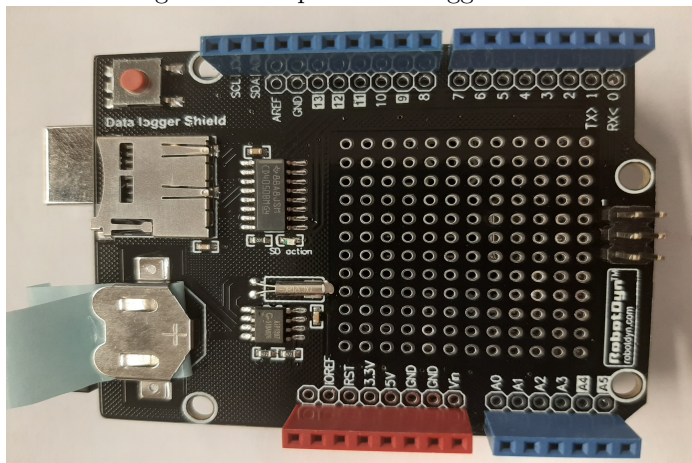
Because shields are designed for a specific functionality the manufactures will often provide example code that utilizes those features. This code can then be modified and/or combined with other code to get your Arduino to do what ever you want. While working with shields be aware of the pins that the shield is using so you don't interfere with its operation. For example pin 9 or 10 will be a special pin for our particular shield that we will use today. You can't use one of these special pins for your instrument because it is already being used for the shield, itself.

7.2 Data Logger Shield

Data logging is a fancy name for storing data. Usually data logging involves the time that the data point is collected as well as the data, itself. And this is often one of the most important parts of any experiment. We will use a data logger shield that has a real time clock (RTC) and a SD card slot built into it. The data logger shield allows us to not only save the data to an SD card but we can also save the time along with it. This is just what we want!

Take a close look at the figure 7.3. You will notice that the SD card slot (right next to the words “Data logger Shield”) is close to an integrated circuit (IC). That IC (a black rectangle) controls the SD card slot.

Figure 7.3: Top of Data Logger Shield



There is also a battery holder. It has a battery in it, but the image also shows a piece of blue paper covering both sides of the battery. This is for shipping so that the battery isn't drained while not in use. The battery is needed for the real time clock so that it keeps time even if there is no other power to the board. If your shield has the blue paper, you can take it out now.

But why do we need a clock on our data logger? Often we just need to know what time the data was collected. But more importantly, we might wish to compare our data to data collected on another instrument. If we know what time the data points were gathered, the comparison is easier. For one example, suppose I am measuring the temperature from the ground to 30km. My instrument measures temperature (like the one we will build today) but I want to know the temperature as a function of altitude. I will need an altimeter to collect data along with my temperature instrument. To match up the data from my instrument and the altimeter, both can record the time for each data point and then I can match the times from my temperature data and the times from the altitude data to get my temperature as a function of altitude. Our BYU-I High Altitude Research Team uses this method for most of their instruments.

On our data logger shield the electronics for the RTC are right next to the battery. Note the smaller IC and the silver cylinder. The cylinder holds a small quartz crystal tuning fork that keeps oscillating. These oscillations are counted by the IC and are used to keep time.

Just like on your Arduino the shield pins are all labeled. But notice that one of our data logger shield types some of the pins are labeled with a white square and a back number. These pins (A4, A5, 9, 11, 12 and 13) are the pins that the shield uses for its functions. Make sure that your experiment doesn't use these pins. Pins A4 and A5 are used for the RTC and pins 9, 11, 12 and 13 are used for the SD card. The bottom of shield gives additional labels to these pins that describe their function.

We have two types of data logger shields. And the second type does not have the nice little white warning boxes to tell you to not use the pins it uses. But most of the pins that you can't use are the same (A4, A5, 10, 11, 12 and 13). The big difference is that for the second type of shield pin 9 got changed to pin 10. These second kind of shield have a brown battery holder.

7.3 Set up

The Data Logger shield uses a library. Libraries are extensions to the computer language we are using. We used one to extend python so we could get data from the serial port (pyserial). We will do something similar to get our data logger shield to work. But this time we have to be very specific. Our real time clock was made by a company named Adafruit. If you think about it, they are the only people who for sure know how the clock works. After all, they designed it! We need to use their real time clock library and no one else's. To get that library follow these steps in the Arduino software.

1. Under "Tools" go to "Manage Libraries ..."
2. Search for "RTCLib"
3. Find the library by Adafruit and install it.

The first time the software is run on the Arduino with the Data Logger shield the RTC needs to be set. Pull out the paper that keeps the battery from contacting and put the battery back (if the paper is still there). Then use the code below in an Arduino program (like the Adafruit RTC example code that they provided with the library!). Check the serial monitor. If it says that the "RTC has not been set!" or if the date and time are incorrect then you will need to remove the comment back slashes on the line

```
rtc.adjust(DateTime(2014, 1, 21, 3, 0, 0));
```

but change the numbers to match the current date and time. Then upload the code again. This will set the clock. This only needs to be done once, as the battery will keep time from then on. So put the comment back slashes back in

and upload the code again. Here is an example data logger code that does all this.

[Download here](#)

```
// =====
// RobotDyn SD/RTC Arduino shield example
// Nolan Chandler
// Last revised: 2020-11-18
//
// Meant to serve as a starting point for data logging
// using both a real-time clock (RTC) and a microSD
// card.
//
// Collects data at a given rate, and saves them with
// a timestamp on the SD card to files called
// "LOG000.TXT", "LOG001.TXT", etc. A new file is
// opened every N samples to collect another N samples.
// =====

#include <SPI.h>
#include <SD.h>

// by Adafruit. Must be manually installed
#include "RTClib.h"

// We have two types of dataloggers. If the battery
// holder is black use pin 9. If the battery holder
// is brown then use pin 10.
const int SD_PIN = 9;

File logfile;

RTC_DS1307 rtc;

float data;
int i = 0;
int N = 50; // Number of samples per file
int waittime_ms = 500; // milliseconds between samples

// =====

void setup()
{
    // Open serial communications
    Serial.begin(9600);
```

```

init_RTC();

// (note 24-hour time: 3pm -> 15)
// This line sets the RTC with an
// explicit date & time, for example:
// to set January 21, 2014 at 3:18pm
// you would use the following line:
// rtc.adjust(DateTime(2014, 1, 21, 15, 18, 0));

init_SD();

logfile = open_next_logfile();
}

// =====

void loop ()
{
  if (i < N) {
    // generate a random number
    data = random(-100, 100);

    DateTime now = rtc.now();

    // Write the date, time and data to log file
    // Same as printing to Serial!
    logfile.print(now.year());
    logfile.print('-');
    logfile.print(now.month());
    logfile.print('-');
    logfile.print(now.day());
    logfile.print('_');
    logfile.print(now.hour());
    logfile.print(':');
    logfile.print(now.minute());
    logfile.print(':');
    logfile.print(now.second());
    logfile.print(',');
    logfile.print(data);
    logfile.println();

    // write same data to serial
    Serial.print(now.year());
    Serial.print('-');
    Serial.print(now.month());
    Serial.print('-');

```

```

    Serial.print(now.day());
    Serial.print('_');
    Serial.print(now.hour());
    Serial.print(':');
    Serial.print(now.minute());
    Serial.print(':');
    Serial.print(now.second());
    Serial.print(',');
    Serial.print(data);
    Serial.println();

    delay(waittime_ms); //ms
    i++;
}

// Reached N samples, open the next log
// file to record N more
else {
    logfile.close();

    // comment out the next two lines to stop
    // recording after the first file
    i = 0;
    logfile = open_next_logfile();
}
}

// =====
// initializes the RTC,
// and checks to see if it has been set
// =====

void init_RTC()
{
    Serial.print("Initializing_RTC...");

    if (!rtc.begin()) {
        Serial.println("_failed!");
        while (1);
    }

    Serial.println("_done!");

    if (!rtc.isrunning())
        Serial.println(

```

```

        "WARNING:_RTC_has_not_been_previously_set");
    }

    // =====
    // attempts to initialize the SD card for reading/writing
    // =====

void init_SD()
{
    Serial.print("Initializing_SD_card...");

    if (!SD.begin(SD_PIN)) {
        Serial.println("_failed!");
        while (1);
    }

    Serial.println("_done!");
}

// =====
// Opens the next available log file in SD:/LOGS/
// Write to the file using logfile.print() or println(),
// just like Serial
// =====

File open_next_logfile()
{
    char filename[24];

    // Create folder for logs if it doesn't already exist
    if (!SD.exists("/LOGS/"))
        SD.mkdir("/LOGS/");

    // find the first file LOGxxx.TXT that doesn't exist,
    // then create, open and use that file
    for (int logn = 0; logn < 1000; logn++) {
        sprintf(filename, "/LOGS/LOG%03d.TXT", logn);

        if (!SD.exists(filename)) {
            Serial.print("Opened_\'SD:");
            Serial.print(filename);
            Serial.println("\'_for_logging.");
            break;
        }
    }
}

```

```
return SD.open(filename, FILE_WRITE);  
}
```

The code above saves random data to the SD card. When you have it working check to see that the data file is on the SD card by putting the SD card in your computer and opening the file. Note that by default it is using pin 9 for the SD card reader/writer. But if you have the data logger shield with the brown battery holder you will need to change the code to use pin 10.

Of course, we don't want random data. You will need to modify the code such that it saves the temperature. But just getting the time information to save to the SD card is a great first step!

Before we go on to add the sensor, spend a few moments looking at the example code. Notes that at the end there are some special functions that do things like initialize the RTC and the SD card reader. These are used by the `setup()` function to get things working. You should not have to change these much. But do note that they need to be part of your code for the data logger shield to work.

Also notice up at the top of the example code there are more include statements and some special variables. Like the

```
RTC_DS1307 rtc;
```

line or the

```
SD_PIN = 9;
```

line. These are also needed by the data logger. So we will have to include them in our data logger codes.

And one more odd thing, there is a whole bunch of code at the end after the loop! These are additional functions that the loop and setup parts of the sketch use. In python, new functions have to come at the beginning of the code because the python interpreter reads the code from the top and won't know about the functions if they come later. But our Arduino code is a variation of C++ and it is compiled before it is sent to the Arduino microcontroller. The compiler looks at all the code before it turns it into microcontroller instructions. So it will find the new functions and use them even though they are at the end of the code file. You could make the temperature algorithm a new function at the end and just use it in the loop if you wish!

You will have to modify the example code. One strategy might be to start with the example code so we are sure we have all of these new parts included. Then add things one at a time to see if the new code breaks the existing code.

7.4 Adding in your sensor

So now that you have your time being recorded (plus a random number) we want to make the modification so that our data logger is recording temperatures. We

will start with the thermistor from your Arduino kit. This thermistor was produced by the Elegoo company that made your kit. It might be tempting to google thermistors to find out how to use it. **But don't do this!** This thermistor was made by a specific company, and we need to go to that specific company to learn how to use it. We might find similar instructions for other thermistors from other companies, but they almost certainly won't be right for our specific thermistor. We need the Elegoo instructions.

And these instructions are hidden on the CD that came with your kit. But who has a CD player in their computer anymore? To get around this we will go to the Elegoo web site and download their instructions. Here is a link:

Link to Elegoo website

Once you are there look for their Support area and choose “Arduino kits files downloads.” For any sensors you get (including the ones for your group design projects) the procedure will be similar. You will need to get information for your Arduino code from the manufacturer of your sensor. We have an UNO R3 starter kit, so choose that option on the website. And download the files for the Super Starter Kit. The Elegoo web site changes from time to time, so you may have to do some hunting. Once you have downloaded the zip file, you will find it includes a PDF file that tells you how to use all of the things in your kit. We want the thermistor. That shows up in “Lesson 15” but there is a catch. Elegoo's lesson 15 has an LCD display that is nice, but not what we want. So you can't just take their wiring diagram without thinking. you need to look for the part that is just for the thermistor. A comparison with Elegoo's Lesson 14 might be helpful, because Lesson 14 shows how to hook up the LCD display. You don't want any of that. So take that away and see what is left! You will see that you need your thermistor and a resistor. The wiring is pretty easy. Does it matter which resistor? Of course it does. We need to follow the manufacturers directions if we want our temperature measurement to work.

What about the code? We know we need to add something to our data logger code. If you look at the Elegoo files you downloaded, you will see example code for all of the lessons. Under Lesson 15 there is code called Thermometer.ino. That looks promising! But we know we will need just the part about measuring temperature, not the part about LCD displays. When you open the code you will find something like this:

```
int tempReading = analogRead(tempPin);
// This is OK
double tempK = log(10000.0 * ((1024.0 / tempReading - 1)));
tempK = 1/(0.001129148+(0.000234125+(0.0000000876741*tempK*tempK))*tempK);
float tempC = tempK - 273.15;           // Convert Kelvin to Celcius
```

This looks like the part that we need! And it fits our model for making a new instrument! We are converting temperature into a voltage using a piece of equipment (the Thermistor) and then using math to convert from voltage back to temperature so we can print the temperature. Of course there must be some physics behind this math. And a good book on thermal resistance could give you that physics. The specific values in the math are usually determined by a curve

fit for the specific thermistor. Because this isn't a class on thermal resistance, we won't study this physics. But we will use the results of other scientists who have by using their mathematical formula to recreate temperature from our voltage measurement.

You will need to add this math into your data logger code. Be careful. I showed the math for the conversion, but you might need other parts of the code (e.g. do you need to set tempPin?). This math should replace the code that makes random numbers in our data logger code. Your challenge is to make this integration of the new and old code happen.

7.5 Digital Data

You might be tempted to complain about how Elegoo built this temperature sensor. Couldn't they have worked a little harder and incorporated all this math into their sensor? Of course we would have to pay more for such a sensor, but if it's not too much more it might be worth it!

The answer is "yes!" And it turns out we have such an upgraded temperature sensor in our kit. It is the DHT11. It is blue and looks like a little blue box attached to a small circuit board.

But what does it look like to get the data already formatted for us? To do this computer engineers have developed ways to transfer data from one piece of electronics to another. We have already learned about one of these, the Universal Serial Bus or USB. But there are other ways of doing this and our DHT11 uses the digital pins on our Arduino to transfer its data. You can see this in the Elegoo Lesson 11. Again we have to go to the manufacturer to find out how to use the new sensor! So we look at the Elegoo documentation. And it has a wiring diagram! The DHT11 needs power and a ground and a digital pin.

The Elegoo data files also include an example code. This time the code is much simpler. There is no LCD display part to ignore. Your job is to integrate this into a new copy of your data logger example code and to save the temperature data to the SD card.

If you have time, it might be good to try to write out the data to the serial monitor and the SD card in a nice format that is easy to read.

7.6 Difficulties with Larger Data Sets and Sensor Libraries

So far logging temperatures has been a challenge, but not too bad. But there can be some difficulties. Arduino microcontrollers have very little memory. Our DHT11 only gives two data points, temperature and humidity. But some sensors have more. The Adafruit BME680 also measures temperature, humidity, atmospheric pressure, altitude, relative humidity, and air quality. Our data loggers can handle all this, but in our Arduino code we have to become much more frugal about memory use. In the DHT11 code we could make the data easy

to understand by putting in words and units in the output. For the BME680 we don't have that privileged. We won't use this today (and depending on your group design project, maybe not at all), but here is an example you can download of a much more memory frugal code for the BME680 sensor.

[Download here](#)

You might have to revisit this as you write codes for our group design projects.

7.7 Lab Assignment

Work in groups of three to five for this set of problems. We have enough equipment for you to each build your own data logger, but work together and don't go on to another step until each team member has completed the previous step.

1. Get the data logger shield up and running and set the correct date and time if needed.
2. Modify the code to take data from a thermistor (included in your kit) and do the math to turn the thermal resistance into a temperature. You will have to look at your Arduino kit manual to know how to write this code. You can start with the example code, but you will have to modify it for the thermistor measurement. Record the temperature on the SD card.
3. Remove the SD card after the data collection is complete, and make sure the data makes sense (compare to a thermometer in the room) and that the SD card writing is working.
4. If there is time, try powering your sensor system on a battery to make sure it can operate independently.
5. If there is time, switch to the digital temperature and humidity sensor. Modify your sketch to read in and output both temperature and humidity values. Again you will have to look at the Arduino kit manual to figure out how to do this. Check your data file to make sure all is working